BACHELOR'S THESIS / ENGINEERING 2025

Automated Invariant-Based PDDL Domain Abstraction for Multi-Agent Classical Planning

Jinsong Liu

upf. Universitat Pompeu Fabra Barcelona

Automated Invariant-Based PDDL Domain Abstraction for Multi-Agent Classical Planning

TREBALL DE FI DE GRAU DE

Jinsong Liu

Director:

Anders Jonsson

Grau en Enginyeria en Informàtica

Academic Year 2025–2026



Acknowledgments

I would like to express my sincere gratitude to my supervisor, Anders Jonsson, whose guidance and supervision were essential throughout the development of this thesis.

Abstract

Cooperative multi-agent systems in classical planning frequently suffer from combinatorial state space explosion. As the number of agents and their available actions increase, the computational cost required to find a coordinated plan grows combinatorially. To mitigate this complexity, this project proposes a fully automated pipeline to abstract PDDL planning domains.

The abstraction process is driven entirely by the generation and analysis of invariant graphs. By identifying and stripping non-essential action constraints, the system first solves a structurally simplified, high-level coordination problem. From this abstract sequence, the pipeline defines and solves per-step subproblems to construct a concrete plan in the original domain. Each abstract step is expanded into a subplan that re-establishes the dropped preconditions before the original action is applied.

This pipeline is implemented in Python and evaluated on the multi-agent assembly domain of Buch Cardona [2024], together with a representative sample of IPC benchmark families.

Resum

Els sistemes multiagent cooperatius en planificació clàssica pateixen sovint una explosió combinatòria de l'espai d'estats. A mesura que augmenta el nombre d'agents i les seves accions disponibles, el cost computacional necessari per trobar un pla coordinat creix de manera combinatòria. Per mitigar aquesta complexitat, aquest projecte proposa un pipeline completament automatitzat per abstreure dominis de planificació PDDL.

El procés d'abstracció es basa íntegrament en la generació i l'anàlisi de grafs d'invariants. Mitjançant la identificació i l'eliminació de restriccions no essencials de les accions, el sistema resol primer un problema de coordinació d'alt nivell estructuralment simplificat. A partir d'aquesta seqüència abstracta, el pipeline defineix i resol subproblemes per pas per construir un pla concret en el domini original. Cada pas abstracte s'expandeix en un subpla que restableix les precondicions eliminades abans d'aplicar l'acció original.

Aquest pipeline està implementat en Python i s'avalua sobre el domini d'assemblatge multiagent de Buch Cardona [2024], juntament amb una mostra representativa de famílies de problemes de l'IPC.

Resumen

Los sistemas multiagente cooperativos en planificación clásica sufren con frecuencia una explosión combinatoria del espacio de estados. A medida que aumenta el número de agentes y sus acciones disponibles, el coste computacional necesario para encontrar un plan coordinado crece de manera combinatoria. Para mitigar esta complejidad, este proyecto propone un pipeline completamente automatizado para abstraer dominios de planificación PDDL.

El proceso de abstracción se basa íntegramente en la generación y el análisis de grafos de invariantes. Mediante la identificación y la eliminación de restricciones no esenciales de las acciones, el sistema resuelve primero un problema de coordinación de alto nivel estructuralmente simplificado. A partir de esta secuencia abstracta, el pipeline define y resuelve subproblemas por paso para construir un plan concreto en el dominio original. Cada paso abstracto se expande en un subplan que restablece las precondiciones eliminadas antes de aplicar la acción original.

Este pipeline está implementado en Python y se evalúa sobre el dominio de ensamblaje multiagente de Buch Cardona [2024], junto con una muestra representativa de familias de problemas de la IPC.

Declaration on the Use of Generative AI

The assistant used was Claude (Anthropic).

In the case of the software, it was used as a coding assistant, with me acting as architect and code reviewer. I consulted it on design alternatives and used it to review implementations, while I set the requirements, took the design decisions, and validated every change throughout development. Some unit tests were written before development to verify the behaviour of the implementation.

In the case of the writing, Claude was used to draft initial snippets of text from a prompt detailing the information to be explained and how, which I then enriched and revised manually, and it was also used to correct prose and grammar. It was also used to translate the abstract from English to Spanish and Catalan.

Contents

1 INTRODUCTION **1**

1.1 Motivation 1

1.2 Contribution 1

1.3 Document Structure 2

2 CLASSICAL PLANNING FUNDAMENTALS **3**

2.1 Classical Planning 3

2.2 Multi-Agent Planning 3

2.3 PDDL and Typing 4

2.4 Fast Downward and LAMA 4

2.5 Assembly Domain 4

2.6 Mutex Groups and Invariant Graphs 5

3 IPC BENCHMARK FAMILIES **7**

3.1 Overview 7

3.2 IPC Families 7

4 METHODOLOGY **9**

4.1 Pipeline Overview 9

4.2 Invariant Graphs 9

4.3 Static Predicate Detection 11

4.4 Abstract Action Construction 11

4.5 Levels of Abstraction 12

4.6 Refinement 13

5 RESULTS AND ANALYSIS **15**

5.1 Experimental Setup 15

5.2 The Assembly Experiment 16

5.3 IPC Results 17

5.4 Domains Where Abstraction Succeeds 18

5.5 Domains Where Abstraction Fails 19

6 CONCLUSIONS AND FUTURE WORK **21**

6.1 Limitations 21

6.2 Future Work 22

List of Figures

2.1 Type hierarchy of the assembly domain. 5
2.2 The two invariant graphs of the assembly domain. 6

4.1 The six-stage pipeline. 9
4.2 The `pick-up` edge of F_{item} and the abstract-action transformation it induces. 12

5.1 Runtime against the number of agents on the assembly scaling grid. 17

List of Tables

5.1 Baseline against the Lotinac pipeline on the assembly scaling grid. 17
5.2 Baseline against the two pipeline routes on the IPC families. 18
5.3 Abstractability, runtime, and plan length on Blocks and ZenoTravel under each invariant route. 19

Chapter 1

INTRODUCTION

Classical Planning is a field in artificial intelligence (AI) concerned with finding a sequence of actions that takes a system from an initial state to a goal state. In multi-agent cooperative problems, the same problem is extended by requiring the actions of several agents to be coordinated so that the joint plan achieves the goal.

This thesis investigates whether the invariants that modern classical planners already discover can be used as a general basis for action abstraction, independently of the planning domain in which they appear. The procedure presented here derives invariant graphs from the Fast Downward mutex finder [Helmert, 2006, 2009] and the invariant tool of Lotinac and Jonsson [2016], and compiles them into an abstract PDDL domain, so the abstract task can be solved by any classical planner and the resulting plan refined back into the original domain. The remainder of this chapter motivates the problem (Section 1.1), states the concrete contributions (Section 1.2), and outlines the document (Section 1.3).

1.1 Motivation

The main challenge in multi-agent cooperative planning is that the complexity of coordinating agents grows combinatorially with the number of agents involved. Each additional agent enlarges the ground action space and the reachable state space that the planner must search, so even modest instances become infeasible to solve directly when several agents must cooperate.

A common remedy is abstraction. The original problem is replaced by a simpler one in which some preconditions or some actions have been removed. The simpler problem is solved with a standard planner, and the resulting abstract plan is then refined into a plan over the original domain. The abstract domain has a smaller search space, so the planner finishes faster, and the refinement step is responsible for filling in the details that were stripped away.

In practice, abstractions are designed by hand for one domain at a time. A domain expert identifies which preconditions can be dropped without breaking the plan, which agent roles can be conflated, and how the refinement should work. This manual process is the main cost of using abstraction in practice, and it limits how broadly the technique applies. This thesis addresses the question by automating the construction of abstractions from a domain's invariant structure rather than from expert decisions.

1.2 Contribution

The main contribution of this thesis is a domain-agnostic procedure that abstracts classical planning domains from the invariants those domains already expose. The procedure builds on two predecessors. From Lotinac and Jonsson [2016] it inherits the lifted invariant graphs and their argument map, and from Buch Cardona [2024] it inherits the per-step refinement strategy. What is new is the automation and the compilation target. The procedure compiles the invariant graphs into an abstract PDDL domain rather than their Hierarchical Task Network (HTN), so any classical planner can solve the abstract task. It schedules a portfolio of abstraction levels, and it characterises which domains the abstraction helps.

The abstract plan is refined back to a plan over the original domain by re-establishing the dropped preconditions before each abstract step. The procedure is evaluated on the multi-agent assembly domain of Buch Cardona [2024] and on a representative sample of IPC benchmark families. On the assembly domain it produces automatically an abstraction that matches the scaling coverage of the hand-crafted abstractions of Buch Cardona [2024], solving every instance of a scaling study, including the instances with many agents on which direct planning exhausts its time budget. The evaluation on the IPC sample characterises the structural property that separates the domains the procedure can abstract from those it cannot.

1.3 Document Structure

The remainder of the document is organised as follows. Chapter 2 fixes the formalism and vocabulary used throughout, including classical planning, multi-agent planning, PDDL, Fast Downward, LAMA, mutex groups and invariant graphs, and the multi-agent assembly domain inherited from the predecessor work of Buch Cardona [2024] that serves as the running example. Chapter 3 presents the IPC benchmark families used to test cross-domain applicability. Chapter 4 describes the abstraction procedure stage by stage, from invariant synthesis through abstract planning to refinement and validation. Chapter 5 reports the empirical evaluation, including the replication of the predecessor result on the assembly domain and the per-domain analysis on the IPC benchmark sample. Chapter 6 summarises the findings and outlines the directions in which the procedure can be extended.

Chapter 2

CLASSICAL PLANNING FUNDAMENTALS

This chapter presents the formalism and the vocabulary used in the rest of the thesis. Section 2.1 defines what a classical planning problem consists of. Section 2.2 extends the formulation to the multi-agent setting in which the methodology is evaluated. Section 2.3 introduces PDDL, the language used to describe domains and problems. Section 2.4 describes Fast Downward, the planner used as the search backend, and the LAMA search variant used throughout the empirical chapters. Section 2.5 presents the assembly domain, the running example used throughout the rest of the thesis. Section 2.6 defines mutex groups and invariant graphs, the structural representation of the domain that the abstraction procedure of Chapter 4 operates on.

2.1 Classical Planning

A classical planning task is described by a domain and an instance, written $P = (D, I)$.

The domain $D = (F, A, T)$ has a set of predicates F, a set of action schemas A, and a type hierarchy T over the types used to declare predicate and action parameters. Each action $a \in A$ has a parameter list par_a, a precondition pre_a, an add list add_a, and a delete list del_a. Predicates and action schemas are written in *lifted* form, with their argument slots as variables. For example, the position predicate of the assembly domain (Section 2.5) has the lifted form `(at_ ?obj ?loc)`. The variables are the *parameters* of the predicate or action, placeholders to be replaced by objects.

The instance $I = (\Omega, s_0, G)$ has a set of typed objects Ω, an initial state s_0, and a goal G. Substituting every parameter of a predicate or action schema with an object from Ω whose type matches the parameter's declared type, or a subtype of it under T, produces the *ground* form. A grounded predicate is called a *fluent*, and a grounded action schema is a ground action. For instance, `(at_ truck1 loc11)` is a well-typed grounding of `(at_ ?obj ?loc)` because `truck1` has type `truck`, a subtype of the parameter's declared type `locatable`. A state s is a set of fluents true at a point in time. An action a is applicable in s when $pre_a \subseteq s$, and applying it produces the state $(s \setminus del_a) \cup add_a$. A plan is a finite sequence of ground actions $\pi = (a_1, \ldots, a_m)$ such that each a_j is applicable in the state reached by $a_1, \ldots, a_{j-1}$ and G is a subset of the final state. The problem of deciding whether such a plan exists is PSPACE-complete in the worst case [Bylander, 1994].

2.2 Multi-Agent Planning

A multi-agent planning task extends the classical formulation of Section 2.1 by introducing several agents that must coordinate to achieve a shared goal. The thesis follows the standard centralised approach, in which a multi-agent task is solved as a single-agent task [Crosby et al., 2014]: agents are encoded as objects of an `agent` type in Ω, and a single centralised planner produces a joint plan in which any action can be executed by any agent whose type matches the action's parameters.

Each action schema with an agent parameter is ground once per agent per binding of its remaining parameters, so the number of ground actions per step grows linearly with the number of agents. The

reachable state space, by contrast, grows multiplicatively: each agent contributes its own private state to the joint state, so if a single-agent task has S reachable states, an n-agent task has up to S^n reachable joint states [Brafman and Domshlak, 2013]. This is a loose upper bound, since the shared environmental state (item positions, road network) is not replicated per agent, but the multiplicative growth in the agent-private factors still drives the practical hardness. With a hundred per-agent states, a two-agent instance has up to 10^4 joint states, a three-agent instance has up to 10^6, and an n-agent instance has up to 100^n. This combinatorial growth is what the abstraction procedure of Chapter 4 aims to reduce.

2.3 PDDL and Typing

PDDL (Planning Domain Definition Language) is the standard formal language for describing classical planning problems. A planning task is split into two files. The domain file declares the predicates of F and the action schemas of A, both in lifted form. The problem file declares the objects of Ω, the initial state s_0, and the goal G.

Each action schema lists its parameters with types, a precondition as a conjunction of fluents, and an effect as a conjunction of fluents whose negated entries are deletes and whose positive entries are adds.

The problem file declares the objects with their types in Ω, the fluents true in s_0, and the conjunction of fluents that must hold in G.

Typing is a convenience that restricts each action schema parameter to objects of the declared type, pruning the grounded action set A_Ω and giving the invariant analysis of Chapter 4 structural information about which objects can occupy which state positions.

2.4 Fast Downward and LAMA

Fast Downward [Helmert, 2006] is the classical planner used as the search backend throughout. Its mutex finder [Helmert, 2009] identifies the lifted invariants of the domain at translation time. The methodology of Chapter 4 consumes that invariant structure directly.

LAMA [Richter and Westphal, 2010] is the search configuration used in every experiment of Chapter 5. It is run in its first-iteration variant (LAMA-first), which returns the first plan found rather than running its iterative-improvement loop.

2.5 Assembly Domain

The assembly domain models a multi-agent task in which a fleet of agents collects items from locations across a map, transports them to workshops, and assembles them into target products. An agent `move`s along the static road network `(path ?l1 ?l2)` between adjacent locations, `pick-up`s an item at its own location, and `drop`s a carried item, the inverse of `pick-up`. Two assemble actions each consume three component items at a workshop and produce a target item, imposing agent-type constraints: `assemble-i3` requires a truck, and `assemble-i4` additionally requires a drone. Agents are partitioned into drones and trucks, as shown in the type hierarchy of Figure 2.1. Listing 2.1 lists the three predicates and the representative `move`, `pick-up`, and `assemble-i3` schemas.

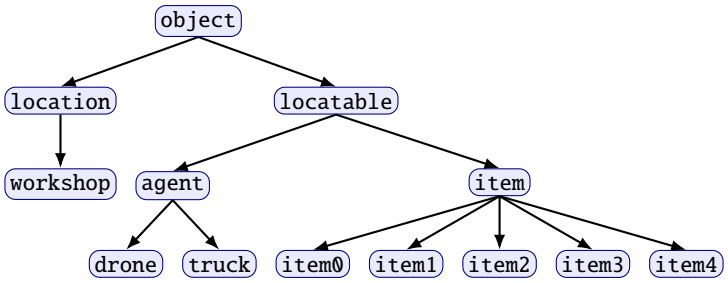


Figure 2.1: Type hierarchy of the assembly domain.

```
(:predicates
   (at_ ?obj - locatable ?loc - location)
   (path ?loc1 - location ?loc2 - location)
   (in ?i - item ?a - agent))

(:action move
   :parameters (?a - agent ?loc1 - location ?loc2 - location)
   :precondition (and (at_ ?a ?loc1) (path ?loc1 ?loc2))
   :effect (and (not (at_ ?a ?loc1)) (at_ ?a ?loc2)))

(:action pick-up
   :parameters (?a - agent ?loc - location ?i - item)
   :precondition (and (at_ ?a ?loc) (at_ ?i ?loc))
   :effect (and (not (at_ ?i ?loc)) (in ?i ?a)))

(:action assemble-i3
   :parameters (?truck - truck ?workshop - workshop
                ?i0 - item0 ?i1 - item1 ?i2 - item2 ?i3 - item3)
   :precondition (and (at_ ?truck ?workshop)
                      (at_ ?i0 ?workshop) (at_ ?i1 ?workshop)
                      (at_ ?i2 ?workshop))
   :effect (and (not (at_ ?i0 ?workshop)) (not (at_ ?i1 ?workshop))
                (not (at_ ?i2 ?workshop)) (at_ ?i3 ?workshop)))
```

Listing 2.1: Predicates and representative action schemas of the assembly domain.

2.6 Mutex Groups and Invariant Graphs

A mutex group is a set of fluents such that at most one fluent of the set is true in any reachable state. A standard example is the location of an object: in a domain where each object occupies one location at a time, the set $\{(\texttt{at obj loc}_1), (\texttt{at obj loc}_2), \ldots\}$ is a mutex group for every fixed `obj`.

Lifted mutex discovery [Helmert, 2009, p. 514] computes mutex groups at the lifted level, where each group is described by a predicate template rather than by an enumerated set of ground fluents. A template is a set of lifted predicates over the parameters introduced in Section 2.1, with one parameter distinguished as bound and the rest as free. The bound parameter indexes the family of mutex groups instantiated by the template, and the free parameters appear inside the fluents of each instance. The algorithm verifies each candidate template against the action schemas in A and the initial state s_0, accepting the template as an invariant when no action schema, under any binding of its parameters, can make two fluents of the template true at the same time.

Invariant graphs [Lotinac and Jonsson, 2016, p. 1276] build on lifted mutex groups by recording how action schemas move the system between the mutually exclusive states of a group. An invariant graph F has k nodes, one per lifted predicate of the underlying mutex template, and an edge from node i to node j

when an action schema in A deletes a fluent of node i and adds a matching fluent of node j under the same binding of the bound parameter. Each edge is described by the tuple (F, i, j, b, a), where a is the action schema contributing the edge and b is the index of the bound parameter in a's parameter list. An action can also touch a node without moving between nodes, by reading its fluent in a precondition, by deleting or adding the fluent of a single node, or by changing only the free arguments of a node's fluent. Each such case contributes a self-loop with $i = j$, and Figure 2.2 shows only the state-changing edges. This tuple notation is the input to the methodology of Chapter 4.

The same lifted predicate can participate in more than one invariant graph when its parameters belong to different types and each choice of bound parameter yields a different family of mutex groups. The assembly domain illustrates this with the predicate `(at_ ?obj ?loc)`, which appears in two invariants that differ in the type of the bound parameter.

- F_{agent}, with $k = 1$ and bound type `agent`. The single state is $s_0 = \{(\texttt{at_}\ \texttt{?a}\ \texttt{?l})\}$ where `?a` has type `agent`. The action `move(?a, ?loc1, ?loc2)` contributes the edge $(F_{\text{agent}}, 0, 0, 0, \texttt{move})$, a self-loop because F_{agent} is single-state.
- F_{item}, with $k = 2$ and bound type `item`. The two states are $s_0 = \{(\texttt{at_}\ \texttt{?i}\ \texttt{?l})\}$ and $s_1 = \{(\texttt{in}\ \texttt{?i}\ \texttt{?a})\}$ with `?i` of type `item`. The action `pick-up(?a, ?loc, ?i)` contributes the edge $(F_{\text{item}}, 0, 1, 2, \texttt{pick-up})$, with bound index 2 because the item is the third parameter, and `drop(?a, ?loc, ?i)` contributes the edge $(F_{\text{item}}, 1, 0, 2, \texttt{drop})$.

This is allowed by the assembly type hierarchy, where `?obj` has type `locatable`, the parent of both `agent` and `item` (Figure 2.1). The two invariant graphs are illustrated in Figure 2.2.

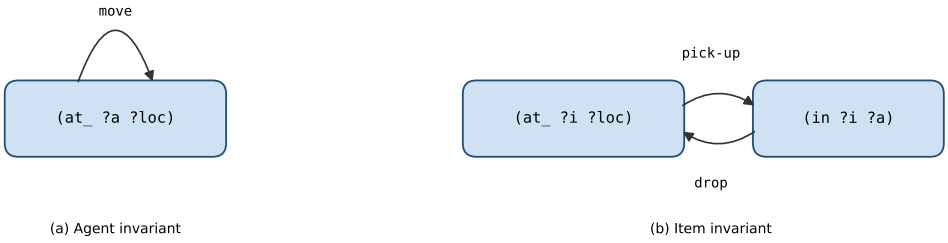


Figure 2.2: The two invariant graphs of the assembly domain.

Chapter 3

IPC BENCHMARK FAMILIES

3.1 Overview

This chapter describes six benchmark families from the International Planning Competition (IPC) corpus vendored alongside the machinery of Lotinac and Jonsson [2016], used together with the assembly domain of Section 2.5 in the empirical evaluation of Chapter 5. The six-family selection exercises different parts of the methodology, including clean multi-state invariants (Blocks, ZenoTravel), static predicates encoding the transition structure (DriverLog, Logistics, Depots), and the single-state invariant of Satellite. The families are summarised by the predicates, actions, and invariant structure relevant to abstraction, rather than reproduced in full.

3.2 IPC Families

Blocks. Models a single arm stacking and unstacking blocks on a table, with predicates `(on ?x ?y)`, `(ontable ?x)`, `(clear ?x)`, `(holding ?x)`, and `(handempty)`. The four actions `pick-up` and `put-down` a block between the table and the arm, and `stack` and `unstack` a block between another block and the arm. The domain has two multi-state invariants per block and no static predicates, and the single-arm restriction is carried by `(handempty)`.

ZenoTravel. Models passengers travelling between cities by aircraft, with positions `(at ?o ?c)`, passengers aboard aircraft `(in ?p ?a)`, and a discretised `(fuel-level ?a ?l)`. Aircraft `board` and `debark` passengers and `fly` or `zoom` between cities, `zoom` trading two fuel levels for one trip, with `refuel` restoring a level. Passengers form a two-state invariant, and the fuel ordering is encoded by the static predicate `(next ?l1 ?l2)` on which the refuelling action depends.

DriverLog. Models drivers walking between locations, boarding trucks, and driving trucks loaded with packages along a road network, with positions `(at ?o ?l)`, loaded packages `(in ?o ?t)`, and a driver in a truck `(driving ?d ?t)`. Drivers `walk` between locations, `board` and `disembark` trucks, `drive` them along the road network, and `load` and `unload` packages. The walking and driving graphs are encoded as static predicates `(path ?l1 ?l2)` and `(link ?l1 ?l2)`.

Satellite. Models a fleet of satellites pointing at directions, calibrating instruments, and taking images. A satellite `turn`s to a direction, `switch`es an instrument on or off, `calibrate`s it, and `take`s an image. The position predicate `(pointing ?s ?d)` is a single-state invariant, and the turning action that changes it carries no static precondition.

Logistics. Models packages moving between cities by truck within a city and by airplane between airports, with positions `(at ?o ?l)` and loaded packages `(in ?p ?v)`. Trucks and airplanes `load` and

unload packages, trucks drive within a city, and airplanes fly between airports. The static predicate (in-city ?l ?c) confines each truck to its containing city.

Depots. Combines logistics and stacking: trucks transport crates between depots and distributors, where hoists stack the crates onto pallets. Trucks drive between places, hoists lift and drop crates, and a truck loads and unloads them. The domain has several invariants, including a Blocks-style (on ?c1 ?c2) stacking group that the goal constrains.

Chapter 4

METHODOLOGY

This chapter presents the procedure that takes a classical PDDL domain and produces an abstract one by compiling invariant graphs, as described in Section 2.6. The invariant graphs come from two routes: the Fast Downward mutex finder [Helmert, 2006, 2009] and the invariant tool of Lotinac and Jonsson [2016]. The procedure additionally uses the Fast Downward planner to solve the abstract task and the plan validator VAL [Howey et al., 2004] to certify each refined plan.

4.1 Pipeline Overview

The pipeline parses a PDDL domain and problem file and produces either a validated concrete plan or a failure report. It runs as six sequential stages, shown in Figure 4.1.

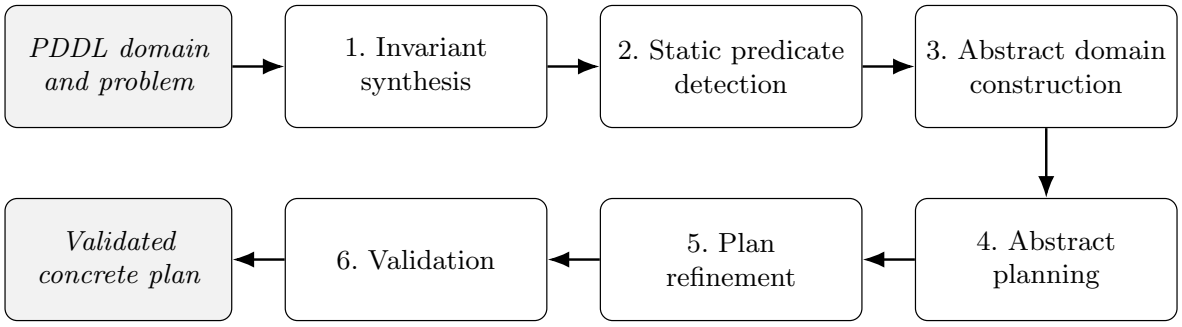


Figure 4.1: The six-stage pipeline.

4.2 Invariant Graphs

The procedure operates on the lifted invariants defined in Section 2.6, illustrated for the assembly domain in Figure 2.2. The set of edges contributed by an action a across all invariants is written $E(a)$.

The invariant graphs are obtained by either of two routes, and the evaluation runs both so that the abstractions they produce can be compared (Section 5.4). The first route invokes the invariant tool of Lotinac and Jonsson [2016]. That tool reasons over the lifted predicates of the domain and emits the lifted invariant graph directly, so the procedure consumes it as is. The second route calls the Fast Downward mutex finder [Helmert, 2009], whose output the procedure lifts into invariant graphs. Neither route covers every benchmark domain: the mutex finder returns no group for the assembly domain, where the tool of Lotinac and Jonsson [2016] is the only source. Because both routes deliver the same kind of lifted invariant graph, the rest of the procedure is identical, and the remainder of this section describes the transformation that the second route requires.

Fast Downward's mutex finder returns a list of invariant groups, one per object of the domain. In each group the bound variable is already grounded to its object, which the procedure identifies as the object

common to the group's fluents. The procedure builds the invariant graphs from these groups in two passes, following the construction of Lotinac and Jonsson [2016, p. 1276] but applied to Fast Downward's mutex output rather than to that tool's own invariants. The first pass produces the nodes: it lifts each group's bound object to its type and merges the groups that share a predicate set and a bound type into a single lifted invariant whose states are the group's predicates, each recording the position of the bound object. On ZenoTravel the group for `person1` is $\{$`(at person1 ?x)`, `(in person1 ?x)`$\}$ and the group for `person2` is $\{$`(at person2 ?x)`, `(in person2 ?x)`$\}$: in each, the instantiated passenger is the object common to both fluents, so it is the bound parameter, and the predicates `at` and `in` are the two states. The groups for the remaining passengers have the same shape, so all per-passenger groups merge into one lifted invariant F_{person} with states $\{$`(at ?p ?c)`$\}$ and $\{$`(in ?p ?a)`$\}$ bound to `?p`, and `?c` and `?a` free.

The second pass produces the edges, which the groups do not carry, from the action schemas. The pass considers each invariant together with each action parameter whose type is the invariant's bound type, a subtype of it, or a supertype of it. Such a parameter contributes an edge with three components:

- the source i is the state whose predicate the action deletes with that parameter at the bound argument,
- the target j is the state whose predicate it adds there,
- the bound index b is the parameter's position in the action.

The type condition is necessary because two invariants can share a predicate at the same position and differ only in their bound type, so a purely syntactic match would attribute the edge to the wrong invariant. The `board` action deletes `(at ?p ?c)` and adds `(in ?p ?a)`, both naming the passenger `?p`, so it contributes the edge from state 0 to state 1 of F_{person}, the tuple $(F_{\text{person}}, 0, 1, b,$ `board`$)$ with b the position of `?p`. A parameter that the action only deletes, only adds, or only requires in its precondition without changing contributes a self-loop. The tuple (F, i, j, b, a) is the bound-parameter slice of the argument map ϕ of Lotinac and Jonsson [2016, p. 1275]: it keeps only the bound parameter, and F's free parameters are not carried by the edge but recovered from a's add and delete lists when the abstract action is built (Definition 4.1).

On the assembly domain, the Fast Downward path does not contribute any abstraction because the mutex finder does not output any invariant groups. The first route skips this two-pass construction: the tool of Lotinac and Jonsson [2016] emits each edge in lifted form, and its output for the `move` action is

```
Invariant [3],[AT_[3,0]];0
Action MOVE[3,0,0]
Inv [3],[AT_[3,0]];0->0
```

Listing 4.1: Lotinac route's output on assembly domain `move` action.

where the integer in brackets is a type index assigned by the tool, invariant `[3]` corresponds to F_{agent}, and `0->0` is the source-target state pair. The procedure parses this into the tuple $(F_{\text{agent}}, 0, 0, 0,$ `move`$)$ defined in Section 2.6, so the rest of the pipeline is identical to the FD route.

In general an invariant can have several bound parameters, one for each object that all the fluents of its group share. The construction takes a single shared object as the bound parameter, so each invariant it builds has one bound parameter. Across the benchmark families every group shares exactly one object, so this choice is unambiguous. An invariant whose group shares several objects, which the general method of Lotinac and Jonsson [2016] permits, would have several bound parameters and is outside the scope of this construction.

In practice the construction yields invariants with $k = 1$, $k = 2$, or $k = 3$ states across the benchmark suite.

4.3 Static Predicate Detection

A predicate p is static in a domain when no action mentions p in any add or delete list, so every grounding of p at the initial state persists for the entire planning episode. Examples include `(path ?l1 ?l2)` in the assembly domain and `(in-city ?l ?c)` in Logistics.

Detection is a single pass over the action schemas. The name of every predicate that occurs in some add or delete effect is collected into a set of dynamic names, and the static predicates are the domain's predicates that this set omits.

The abstract action construction (Section 4.4) chooses whether to retain the static preconditions of each abstract action via a binary flag. The two settings, *drop-statics* and *keep-statics*, are interleaved by the level sequence of Section 4.5.

4.4 Abstract Action Construction

Definition 4.1 (Abstract action)**.** *Given an action a with edge set $E(a)$, the* abstract action *of a is constructed as follows.*

If $E(a) = \emptyset$, the action a is not abstracted at all. It passes through unchanged, keeping its original precondition, effect, and name, so no abstract action is produced for it.

Otherwise, the abstract action has the same parameter list as a. Each edge $(F, i, j, b, a) \in E(a)$ contributes two fluents drawn from a's own precondition and effect, each carrying a's b-th parameter at the bound argument. The source fluent, taken from a's precondition or delete list for state s_i, becomes both a precondition and a delete effect. The target fluent, taken from a's add list for state s_j, becomes an add effect. A source fluent is omitted when it coincides with the target fluent, which is a self-loop that does not change state, or when the action does not delete it, which happens when the matched fluent is a precondition the action reads but leaves in place.

When keep-statics mode is selected by the procedure of Section 4.5, every static predicate in a's original precondition is added as a precondition of the abstract action. The static predicates are the set S identified in Section 4.3.

The matching procedure of Definition 4.1 is the argument map ϕ of Lotinac and Jonsson [2016], derived directly from the action. The abstract effect is built from the edge targets and sources alone, so any original effect on a predicate that no kept invariant covers is discarded. This is the design decision behind relaxations such as dropping an uncovered `(handempty)` delete, and the completeness it can cost is absorbed by the level fallback of Section 4.5. As a first worked example, Definition 4.1 is applied to the assembly action `pick-up(?a, ?loc, ?i)` from Section 2.5. Its only state-changing edge is $(F_{\text{item}}, 0, 1, 2, \texttt{pick-up})$. The source fluent from $s_0 = \{(\texttt{at_ ?i ?l})\}$ instantiates to `(at_ ?i ?loc)` and the target fluent from $s_1 = \{(\texttt{in ?i ?a})\}$ instantiates to `(in ?i ?a)`. The source and target differ and the original action deletes the source, so the rule adds the source as both a precondition and a delete effect, and the target as an add effect.

```
(:action abstract_pick-up
    :parameters (?a - agent ?loc - location ?i - item)
    :precondition (at_ ?i ?loc)
    :effect (and (in ?i ?a) (not (at_ ?i ?loc))))
```

Listing 4.2: `abstract_pick-up`.

Figure 4.2 illustrates the transformation on the F_{item} graph.

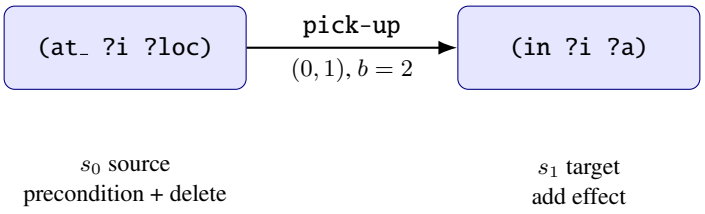


Figure 4.2: The `pick-up` edge of F_{item} and the abstract-action transformation it induces.

A second worked example shows how the rule handles a single-state invariant. The assembly action `move(?a, ?loc1, ?loc2)` contributes the edge $(F_{\text{agent}}, 0, 0, 0, \texttt{move})$. The bound parameter is the agent `?a`. The source fluent from $s_0 = \{(\texttt{at_ ?a ?l})\}$ instantiates to `(at_ ?a ?loc1)` and the target fluent from the same s_0 instantiates to `(at_ ?a ?loc2)`. Although $i = j$, the source and target fluents differ in their free argument (`?loc1` versus `?loc2`), and the original action deletes the source. The rule therefore adds the source as both a precondition and a delete effect.

```
(:action abstract_move
    :parameters (?a - agent ?loc1 - location ?loc2 - location)
    :precondition (at_ ?a ?loc1)
    :effect (and (at_ ?a ?loc2) (not (at_ ?a ?loc1))))
```

Listing 4.3: `abstract_move` under drop-statics.

The static precondition `(path ?loc1 ?loc2)` is dropped under drop-statics because it is not the source of any edge, so the abstract `move` can transition the agent between any two locations. Refinement cannot reconnect such a step: it re-imposes the original `move`'s full precondition, including the static `(path ?loc1 ?loc2)`, which no action can establish, so a non-adjacent abstract `move` is unrefinable and the level fails. This is the movement failure analysed in Chapter 5. Under keep-statics the static is re-added and the abstract action coincides with the original `move`.

The construction transforms only the action set. The abstract domain reuses the original predicates, types, and constants, so the problem file is passed to the abstract planner with its initial state and goal unchanged. Because the abstract action keeps the full parameter list, dropping a precondition reduces the constraint structure but can enlarge the grounded action set. The abstract `move`, for instance, grounds over all location pairs rather than the adjacent ones alone.

4.5 Levels of Abstraction

The methodology is attempted at several levels of abstraction, each defined by a combination of *keep_statics* and a set of invariant graphs skipped from $E(a)$. The pipeline tries the levels in a fixed order, from the broadest abstraction to progressively more conservative ones, and stops at the first that yields a refined plan. The most conservative level retains a single active invariant graph. If even that level does not yield a refined plan, the problem is reported as not abstractable by the methodology.

```
level 1:   keep_statics = false, skip no graph
level 2:   keep_statics = true,  skip no graph
level 3:   keep_statics = true,  skip the largest invariant graph
level 4:   keep_statics = true,  skip the two largest invariant graphs
...
level N-1: keep_statics = true,  skip all but one invariant graph
```

Listing 4.4: The sequence of abstraction levels.

The largest invariant graph is the one with the most nodes, with ties broken by a stable identifier, and the sequence ends at the level that keeps a single graph active.

The first two levels are the static-precondition policy. The drop-statics level drops every static predicate from the abstract preconditions, and if that attempt fails by either Fast Downward returning no abstract plan or refinement reporting a failure, the procedure falls back to the keep-statics level which retains the statics. In some benchmark domains, the static predicates are what restrict an action to its concrete-domain transitions. Assembly's `(path ?from ?to)` restricts `move` to adjacent locations. Logistics' `(in-city ?l ?c)` restricts `drive-truck` to within a single city. Keeping such statics in the abstract precondition reproduces the restriction, so the abstract action is identical to the concrete one and the abstraction has no effect. Dropping the statics removes the restriction and produces the relaxation that the abstraction is meant to deliver.

The level at which a problem first succeeds indicates how much invariant structure the abstraction must retain for refinement to reconstruct a valid plan. A problem solved at level 1 tolerates the broadest abstraction, and one solved only at the last level tolerates abstraction over a single invariant graph. Problems for which no level produces a refined plan are reported as not abstractable.

Per-domain distributions of the level at which problems first succeed, together with the share of problems reported as not abstractable, are reported in Chapter 5.

4.6 Refinement

Fast Downward is invoked against the abstract domain and returns a plan that is a sequence of abstract and non-abstracted actions (the abstract planning stage of Section 4.1). To produce a valid plan in the original domain, a refinement step expands every abstract action into a subplan that re-establishes the action's original preconditions, followed by the original action itself. Actions that were not abstracted are kept as they are. This refinement strategy follows the predecessor procedure of Buch Cardona [2024, p. 11].

A simulated concrete state is kept while iterating over the abstract plan. For each step:

1. If the step's action is from the original domain, no refinement is needed. The action's effects are applied to the simulated state and the step is appended to the refined plan as is.
2. If the step's action is abstract, a subproblem is built with the current simulated state as its initial state and the grounded preconditions of the corresponding *original* action as its goal. The subproblem is solved by Fast Downward against the original domain. The resulting subplan is appended to the refined plan, followed by the original action with the abstract step's arguments. The simulated state is advanced by applying the effect of every appended action.

By Definition 4.1, an abstract action's precondition is a subset of the original action's precondition. The subproblem goal is the original action's full precondition. The literals the abstraction kept already hold in the simulated state, so the subplan only has to re-establish the ones the abstraction dropped before the original action is applied. This refinement is greedy: each subproblem is solved independently, so re-establishing one step's preconditions can undo a literal that a later step or the goal requires, and the refiner neither reorders steps nor backtracks to repair such interactions. The refined plan is therefore checked with VAL at the validation stage of Section 4.1, and a plan that VAL rejects or that leaves a goal fluent unestablished is reported as a failure. As an example, consider the abstract pick-up of Listing 4.2. An abstract plan containing the step `(abstract_pick-up truck1 ws1 i1)` triggers a subproblem with the current simulated state as init and the grounded preconditions of the original pick-up, `(at_ truck1 ws1)` and `(at_ i1 ws1)`, as goal. The subproblem is solved against the original domain, and Fast Downward returns a subplan such as `(move truck1 loc11 ws1)` that brings the truck to the workshop where the item already sits. The original pick-up `(pick-up truck1 ws1 i1)` is then appended to the refined plan, and applies with both of its preconditions satisfied.

Per-step precondition checking is delegated to VAL [Howey et al., 2004], which replays the refined plan step by step against the original domain and accepts it only if the precondition of each action holds at the

point of application and the goal of the problem holds in the final state.

Soundness. Because VAL runs against the original domain, acceptance certifies that the refined plan is a sound plan for the input problem, independently of the abstraction.

A plan that VAL rejects is treated as a failure of the current abstraction level, which triggers the next attempt in the procedure of Section 4.5.

Chapter 5

RESULTS AND ANALYSIS

This chapter evaluates the methodology of Chapter 4 on the multi-agent assembly domain of Buch Cardona [2024] and on six families from the IPC corpus. The primary metric is the *abstractability rate*, the fraction of instances on which the pipeline returns a VAL-valid plan whose construction required the refiner to expand at least one abstract action into a subplan. The number of actions the refiner inserts to re-establish the dropped preconditions is recorded per instance, and an instance counts as abstractable only when this count is positive. A zero count means the refined plan is identical to the plan found in the abstract domain, which happens either when the level sequence has fallen back to the original domain or when the dropped preconditions happened to already hold so the abstraction made no difference to the plan. A positive count is the signal that the abstract domain both differed from the original and mattered to the plan.

Coverage is the fraction of instances on which the pipeline returns a sound plan, with or without abstraction. On the IPC families coverage is near-complete and carries little information: Fast Downward already solves all but two Depots instances within the time budget, and the level sequence falls back to the original domain whenever no abstraction applies, so the pipeline inherits that coverage by construction.

Section 5.1 describes the experimental setup, including how the assembly instances are generated. Section 5.2 reports the assembly experiment and the replication of the predecessor results of Buch Cardona [2024]. Section 5.3 summarises the IPC families under both invariant routes against the baseline. Section 5.4 then reports the families where the abstraction succeeds, and Section 5.5 those where it does not.

5.1 Experimental Setup

The experiments are run on a machine with an AMD Ryzen 7 7800X3D processor (8 cores, 16 threads) and 64 GB of RAM, running Linux with Python 3.14. The experiments use the three external tools introduced in Chapter 4. Fast Downward is run in its first-iteration LAMA configuration on each abstract problem and refinement subproblem. This planner setup mirrors that of Buch Cardona [2024], so the assembly scaling result reported in Section 5.2 is a direct replication with abstraction as the only variable. Each call to Fast Downward is given a wall-clock budget of 60 seconds on the IPC experiment and 600 seconds on the assembly experiments. The baseline makes a single call, while a pipeline run makes one call for abstract planning and one per refinement step at each level it attempts, so the total time of a pipeline run can exceed the per-call budget.

The IPC experiment covers 145 instances across six families: Blocks (35 instances), ZenoTravel (20), DriverLog (20), Satellite (20), Logistics (28), and Depots (22). These instances are taken unchanged from the benchmark set distributed with the implementation of Lotinac and Jonsson [2016]. The assembly experiment replicates the scaling study of the predecessor work [Buch Cardona, 2024]. The scaling instances are produced by a generator contributed by this thesis, which takes the number of locations, workshops, drones, trucks, and items per type, and the number of `assemble-i3` and `assemble-i4` goals as parameters. The location graph is constructed by drawing a spanning tree followed by random extra edges to a target density, so the resulting graph is always connected.

Each problem is run under three configurations. The baseline runs Fast Downward directly on the original

domain, with no abstraction. The other two are the pipeline of Chapter 4 fed by the two invariant routes of Section 4.2: one takes its invariant graphs from Fast Downward's mutex finder, the other from the invariant tool of Lotinac and Jonsson [2016]. The two pipeline configurations share the same level sequence and differ only in the source of the invariants. Neither route covers every domain: Fast Downward returns no group for the assembly domain, and the tool of Lotinac and Jonsson [2016] does not support Satellite. The Fast Downward route is therefore run on the six IPC families, the Lotinac route on the assembly domain and the five IPC families it supports, and the baseline on all of them. Section 5.4 compares the two routes where both abstract.

For each pipeline run the level sequence of Section 4.5 is executed, and the level at which an abstract domain first produces a refined plan that VAL certifies is recorded. The runtime reported for the baseline is the time Fast Downward takes on the original problem, and the runtime reported for a pipeline run is the time it takes on the abstract problem and the refinement subproblems, including the earlier levels that the pipeline attempts and abandons before one succeeds. Per-route ranges in the IPC results aggregate only the instances each route abstracts, so the baseline and pipeline numbers in a given row refer to the same set of instances. The implementation, the benchmark instances, and the per-domain CSV outputs are available at https://github.com/WdnLiu/TFG.

5.2 The Assembly Experiment

The assembly experiment replicates the scaling study of the predecessor work [Buch Cardona, 2024].

The abstract domain that the pipeline produces on the assembly domain comes from the invariant tool of Lotinac and Jonsson [2016], since Fast Downward's mutex finder returns no group there.

The abstraction it produces is weaker than the hand-crafted ABS2 of Buch Cardona [2024], which removes `move` and strips the carrier position from the assemble actions, but stronger than their ABS1. The level the pipeline reaches is 2. Level 1, which drops the static predicates, fails to refine: without the static `(path ?l1 ?l2)`, `abstract_move` becomes a teleport between non-adjacent locations that refinement cannot reconnect, since no action can re-establish a static. The pipeline therefore falls back to level 2, where the statics are kept. The assemble actions contribute no edge to any invariant, so the abstraction rule leaves them unchanged by its base case and they keep their full precondition, including `(at_ ?truck ?workshop)`. At this level `move` also retains its static `path`. The relaxation is therefore confined to `pick-up` and `drop`, which lose the agent co-location precondition `(at_ ?a ?loc)`. The abstract planner therefore picks up and drops items without routing an agent to each one, and refinement re-establishes the co-location by routing an agent before every concrete `pick-up` and `drop`, while the carriers are still moved to the workshop for assembly in the abstract plan itself.

The scaling study follows the predecessor's setup: 50 and 100 locations crossed with 2, 4, 8, 16, and 32 agents, for a total of 10 instances, with a 600-second wall-clock budget per call. The per-instance figures are given in Table 5.1. The pipeline returns a VAL-valid refined plan on all 10 instances. Direct Fast Downward returns a plan on 5 instances and fails on the remaining 5, at 16 and 32 agents in the 50-location group and at 8, 16, and 32 agents in the 100-location group. The three 100-location failures exhaust the 600-second budget, while the two 50-location failures terminate without a plan before the budget is reached. The pipeline's runtime grows gently with the agent count, from about 8 to about 110 seconds as the agent count grows from 2 to 32, while direct Fast Downward's runtime rises steeply until it fails to return a plan at all (Figure 5.1). The methodology therefore matches the predecessor coverage on every instance and dominates direct Fast Downward as the agent count grows.

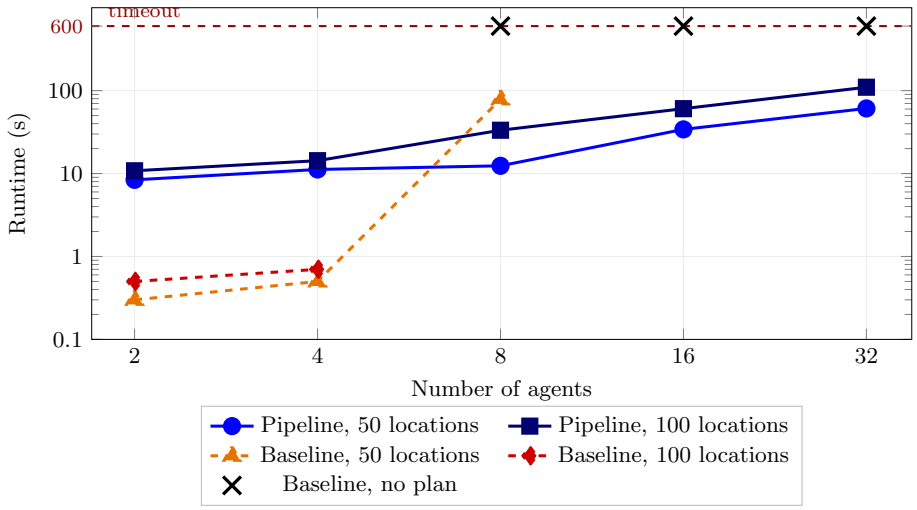


Figure 5.1: Runtime against the number of agents on the assembly scaling grid.

Table 5.1: Baseline against the Lotinac pipeline on the assembly scaling grid.

		Baseline		Lotinac pipeline		
Locations	Agents	Time (s)	Plan length	Time (s)	Abstract length	Refined length
50	2	0.3	49	8.4	20	48
50	4	0.5	54	11.2	20	53
50	8	78.5	50	12.4	19	49
50	16	—	—	34.1	19	49
50	32	—	—	61.0	19	56
100	2	0.5	49	10.8	20	48
100	4	0.7	58	14.3	19	46
100	8	—	—	33.3	20	73
100	16	—	—	60.6	21	67
100	32	—	—	109.9	19	65

5.3 IPC Results

The pipeline was run on the six IPC families under both invariant routes of Section 5.1: the Fast Downward mutex finder, and the invariant tool of Lotinac and Jonsson [2016]. Table 5.2 compares direct Fast Downward, the baseline, against the two pipeline routes. For the baseline it gives the fraction of instances solved within the budget and the mean runtime. For each route it gives the abstractability rate, the fraction of instances on which the refiner had to splice in at least one subplan, and the mean runtime. On these families the baseline already solves almost everything in well under a second, so the pipeline does not improve on it: its value is on the assembly domain of Section 5.2, where direct planning fails at scale. The Lotinac route does not apply to Satellite, whose `(=)` precondition its parser rejects. The two routes abstract different families: Section 5.4 and Section 5.5 analyse, family by family, why each does or does not pay off.

Table 5.2: Baseline against the two pipeline routes on the IPC families.

	Baseline		FD mutex finder		Lotinac	
Domain	Solved	Time (s)	Abst.	Time (s)	Abst.	Time (s)
Blocks	35/35	0.2	35/35	7.3	0/35	5.8
ZenoTravel	20/20	0.6	14/20	35.8	19/20	28.8
DriverLog	20/20	0.7	1/20	18.8	1/20	14.1
Logistics	28/28	0.2	0/28	5.3	0/28	6.5
Depots	20/22	10.2	0/22	357.6	0/22	24.0
Satellite	20/20	0.3	0/20	0.6	—	—

5.4 Domains Where Abstraction Succeeds

Two IPC domains abstract a substantial fraction of their instances: Blocks and ZenoTravel. Each shows a different reason the abstraction does or does not pay off.

- **Blocks** (Section 3.2, $35/35$). The abstraction strips the single-arm `(handempty)` precondition from every pickup, treating the arm as an unbounded-capacity gripper. It abstracts cleanly on every instance because the stacking order is barely affected by whether the arm holds one block or several. Blocks is a single-arm, single-agent problem, so there is no coordination for the abstraction to remove, and Fast Downward solves the original in a fraction of a second.
- **ZenoTravel** (Section 3.2, $14/20$). The abstraction strips the aircraft-position precondition from `board` and `debark`, so the abstract plan boards and debarks passengers without flying the aircraft to them, and refinement re-inserts the flights by planning each gap against the original domain, which still enforces fuel. Seven of the twenty instances solve at the drop-statics level. On a further seven, drop-statics also removes `fly`'s fuel static `(next ?l2 ?l1)`, the predicate that forces each flight to change fuel by exactly one level. Without it the abstract planner takes a multi-level jump such as `fl4` straight to `fl0`, and the refiner cannot re-establish the static `(next fl0 fl4)` this implies, so these seven fall back to keep-statics, where `(next)` is retained and every flight again changes fuel by one level. The remaining six instances are not abstractable: their goal fixes an aircraft's city, which the greedy refiner re-establishes step by step but never plans toward, so they fall back to retaining the aircraft graph and the abstraction makes no difference. Every instance is solved, but each refinement gap is a separate planner call, so the pipeline takes far longer to return a plan than the baseline, as Table 5.3 reports.

Even where the abstraction returns a valid plan, it does not reduce runtime on the IPC instances, which are small enough for Fast Downward to solve directly. Table 5.3 reports the abstractability rate, the runtime of the baseline and the pipeline, and the length of the baseline plan, the abstract plan, and the refined plan for Blocks and ZenoTravel. The abstract plan is shorter than the baseline because it omits the positioning steps the abstraction removes, and refinement restores them, so the refined plan is comparable to the baseline on Blocks but consistently longer on ZenoTravel, where peak refined length exceeds peak baseline length by up to 36% under either route. In runtime, the baseline stays under two seconds while the pipeline rises with the cost of the extra planning pass and the refinement, so the pipeline is the slower of the two throughout.

The two invariant routes disagree on exactly these two domains, and in opposite directions. On Blocks, Fast Downward's mutex finder leaves `(handempty)` outside every group, so the abstraction drops it and the single-arm relaxation applies to all thirty-five instances. The tool of Lotinac and Jonsson [2016] instead groups `(handempty)` with `(holding)` and leaves `(clear)` uncovered, so it drops `(clear)`, but the abstract planner does not take advantage of the relaxation: every pick-up in its plans happens to

Table 5.3: Abstractability, runtime, and plan length on Blocks and ZenoTravel under each invariant route.

		Runtime (s)		Plan length		
Domain	Abstractable	Baseline	Pipeline	Baseline	Abstract	Refined
Blocks (FD)	35/35	0.1–0.2	1.1–27.8	6–240	6–148	8–234
ZenoTravel (FD)	14/20	0.2–2.7	2.0–137.5	9–104	5–50	11–142
ZenoTravel (Lotinac)	19/20	0.2–2.7	1.8–124.6	6–104	5–75	9–142

target a clear block anyway, so refinement inserts no subplan and no instance counts as abstractable. On ZenoTravel the advantage reverses. Fast Downward synthesises a separate `(fuel-level)` invariant, so its abstract `fly` keeps the fuel-level precondition unchanged and the relaxation comes only from `board` and `debark`. The tool of Lotinac and Jonsson [2016] synthesises no fuel-level group, so it also drops `(fuel-level)` from `fly`: the abstract plan flies without matching the current fuel level and refinement restores it, which lifts the abstractability from fourteen to nineteen of the twenty instances, at the cost of a longer refined plan, as Table 5.3 reports. Neither decomposition dominates, so the effectiveness of the method depends on the quality of the invariant decomposition and not only on its validity: two correct decompositions of the same domain can yield very different abstractions, one substantially helping the planner while the other has no practical effect.

The assembly domain benefits because the abstraction converts a tightly coupled multi-agent problem into a series of independent single-agent subproblems. Dropping agent positions lets the abstract planner decide only which agent assembles what, and refinement plans each agent's route separately, so the search space is far smaller than the original joint problem in which every agent's movement is interleaved. ZenoTravel and Logistics are multi-agent too, but Fast Downward already solves their instances in well under a second, so even where the abstraction applies it removes no coordination cost. The runtime benefit therefore comes from decoupling agents that must otherwise be coordinated, not from the presence of multiple agents.

That the same invariant-based construction produces a genuine abstraction across several unrelated domains, including the multi-agent assembly domain, shows that the method is not specific to the assembly domain it was derived from. The runtime benefit, by contrast, is realised only in the regime the assembly scaling experiment of Section 5.2 reaches: a tightly coupled multi-agent task whose joint search defeats the planner but whose decoupled subproblems do not.

5.5 Domains Where Abstraction Fails

Four IPC families abstract few instances or none at all, for reasons that differ by domain. Logistics and Satellite fail for clear structural reasons, given below. For DriverLog, Depots, and the six unabstracted ZenoTravel instances of Section 5.4, the symptom is the same at the level that keeps the static predicates: the refined plan leaves a goal fluent unsatisfied, a driver, a crate, or an aircraft that does not reach its goal configuration. The per-step refiner of Section 4.6 re-establishes the preconditions of each abstract step locally but does not plan toward the goal, so when the abstraction relaxes a configuration that the goal constrains, the refined plan does not always restore it. No level of the sequence then yields a valid non-trivial abstraction, and the pipeline falls back to a trivial level.

DriverLog. (Section 3.2, 1/20.) The abstraction drops the driver and truck positions, and the failure is the one described above: the goal pins drivers and trucks to final locations that the greedy refiner never plans toward, so the refined plan leaves those goal fluents unsatisfied. Nineteen instances therefore fall back to a trivial level, and only the smallest instance abstracts.

Logistics. (Section 3.2, $0/28$.) The static `(in-city)` that confines a truck to its city appears only on `drive-truck`, not on `load-truck` or `unload-truck`. When the abstraction strips the truck-position precondition from load and unload, the abstract plan moves packages between cities by loading and unloading them directly, never invoking `drive-truck` and so never meeting `(in-city)`. Refinement cannot realise these moves, because a single truck cannot leave its city, so the aggressive levels produce no valid plan and the pipeline falls back. The abstraction has nothing to offer in the other direction either: within a city a truck already moves between any two locations in one `drive-truck` step with no adjacency constraint, so the relaxation that lets `move` teleport in the assembly domain has nothing to relax here.

Satellite. (Section 3.2, $0/20$.) The predicate `(pointing ?s ?d)` is a single-state invariant whose only transitions are self-loops, since `turn_to` changes the direction within the single pointing state rather than moving between states. The abstraction has no inter-state transition to exploit, and at the aggressive levels it produces an abstract problem that Fast Downward reports unsolvable, so the level sequence falls back to the original domain on every instance.

Depots. (Section 3.2, $0/22$.) The domain has multiple goal-relevant invariants, including a Blocks-style `(on ?c1 ?c2)` stacking group, and abstracting any one risks producing refined plans whose stacking goals cannot be re-established. No instance abstracts: at every non-trivial level the per-step subplanner cannot re-establish a precondition or the goal stacking fluent is left unsatisfied after refinement, so the sequence falls back to the original domain throughout.

Taken together, these four families exhibit three structurally distinct failure patterns, and show that the methodology does not generalise as a single uniform procedure. Logistics offers no restriction worth relaxing. Satellite offers no inter-state transition to abstract. DriverLog and Depots relax a predicate the goal depends on, so the refined plan leaves a goal fluent unsatisfied. Because each obstacle is different, no single adjustment to the abstraction rule removes all of them. Generalising the methodology would require analysing the invariant structure of each domain more deeply before abstraction, rather than applying the uniform lifted rule used here. The directions this analysis would take are set out in Section 6.2.

Chapter 6

CONCLUSIONS AND FUTURE WORK

This thesis has evaluated a method for abstracting PDDL domains automatically from the invariants a domain already exposes, drawing the invariant graphs from two sources: the Fast Downward mutex finder, and the invariant tool of Lotinac and Jonsson [2016]. The objective was to investigate whether useful action abstractions can be constructed automatically rather than by hand. The work continues that of Buch Cardona [2024], who evaluated hand-crafted abstractions of the multi-agent assembly domain to address its complexity and scalability. The pipeline presented here reproduces the success of that hand-crafted abstraction automatically, computing plans for assembly instances on which direct Fast Downward fails.

The methodology does not, however, generalise to every domain. Across the IPC families, the cases where the pipeline produces no abstraction fall into three structurally distinct patterns: a static predicate that encodes the reachability of the invariant's bound parameter, a goal that references a fluent the invariant tracks, and a single-state invariant on which every action self-loops. The first makes concrete the trade-off between compiling to PDDL and to a Hierarchical Task Network (HTN). The achieve-q recursion of Lotinac and Jonsson [2016] resolves it inside HTN, but the PDDL compilation adopted here has no equivalent construct. The goal-dependence pattern is the same problem: the abstraction drops a precondition the goal eventually requires, which PDDL cannot recursively re-establish. The single-state pattern is different: Satellite's invariant has only one state, so the abstraction rule finds nothing to relax.

A second finding is that the pipeline abstracts both single-agent and multi-agent domains, and that the gain on multi-agent domains comes from decoupling. As in Buch Cardona [2024], the abstraction transforms a tightly coupled multi-agent problem into a series of single-agent subproblems, so refinement plans each agent's route independently and instances on which Fast Downward struggles alone become feasible.

A third finding is that a successful abstraction does not by itself guarantee a faster solution. On the IPC families the abstraction frequently applies, yet Fast Downward already solves these instances in well under a second, so the additional abstract planning and refinement passes leave the pipeline slower than the baseline. The runtime benefit appears only when the original task is hard enough to defeat direct planning, which in this evaluation means multi-agent coordination rather than the mere presence of several agents.

A final finding is that the abstraction depends on the invariant decomposition it is given. The two sources produce different but equally valid groupings, and they disagree on which domains abstract: the Fast Downward mutex finder abstracts Blocks where the tool of Lotinac and Jonsson [2016] does not, and the reverse holds on ZenoTravel. Two correct decompositions of the same domain can therefore yield very different abstractions, one substantially helping the planner while the other has no practical effect, so the quality of the decomposition, not only its validity, governs whether the abstraction helps.

6.1 Limitations

The PDDL compilation cannot express achieve-q. The methodology compiles invariant graphs into an abstract PDDL domain in which a dropped precondition is left unresolved at the abstract level and re-established at refinement time. PDDL has no construct equivalent to the achieve-q recursion of the HTN compilation in Lotinac and Jonsson [2016], which would enumerate possible bindings of the dropped precondition through a subtask. The structural-unreachability failures of Section 5.5 are the cost of this

choice.

Binary static-precondition policy. The level sequence treats statics as a single bulk, keeping every static across the domain or dropping every static. A finer policy that keeps the statics required by particular actions while dropping others would give the level sequence more intermediate points between full relaxation and the original domain, at the cost of a longer sequence.

6.2 Future Work

Backtracking refiner with achieve-q semantics. The highest-impact extension replaces the greedy per-step refiner with a backtracking refiner that enumerates binding choices for each dropped precondition and retries when subplanning fails. This brings the behaviour of the methodology closer to the achieve-q recursion of the HTN compilation in Lotinac and Jonsson [2016] and would address most of the structural-unreachability failures reported in Section 5.5.

Dropping statics at intermediate levels of abstraction. The binary keep-statics policy of Section 4.5 keeps or drops every static at once. Replacing it with a sequence that relaxes statics in subsets, ordered by their participation in invariant edges, would give the level sequence intermediate points between the fully relaxed and the original domain, and would let a domain keep the statics its refinement depends on while relaxing the rest.

Goal-relevance pruning of irrelevant structure. The abstraction rule only relaxes preconditions and never removes an action. A goal-directed pruning step, applied after the rule, could discard invariants whose bound type is unreachable from the goal objects, along with the actions and preconditions that concern only that goal-irrelevant structure. This is an instance of relevance analysis in planning [Nebel et al., 1997] and is orthogonal to the invariant rule. On the assembly domain it would remove the `move` action outright, reaching a more aggressive abstraction than the invariant rule reaches on its own. It is left out of the present methodology because it is a different abstraction mechanism, goal-relevance rather than invariant structure, and so falls outside the invariant-based scope of this thesis.

Broader benchmark evaluation. The evaluation covers six IPC families and the multi-agent assembly domain. Extending it to the remaining IPC families would test whether the three-pattern characterisation of structural unabstractability generalises beyond the sample reported in Section 5.5. The runtime benefit, tied to multi-agent coordination, would be tested most directly on dedicated multi-agent benchmarks such as the CoDMAP competition set, which are written in factored multi-agent PDDL and would first require compiling each task into the single centralised STRIPS task the methodology operates on. The abstraction itself is planner-agnostic, so rerunning the pipeline with other classical planners, or with Fast Downward in configurations other than LAMA-first, would test how the runtime benefit depends on the search engine the abstraction feeds into, and how much of the refined plan length growth visible in Table 5.3 comes from satisficing search rather than from the per-step refinement strategy.

Bibliography

Brafman, R. I. and Domshlak, C. (2013). On the complexity of planning for agent teams and its implications for single agent planning. *Artificial Intelligence*, 198:52–71.

Buch Cardona, A. (2024). Abstractions for multi-agent planning scalability. Bachelor's thesis, Universitat Pompeu Fabra. Supervisors: Anders Jonsson and Javier Segovia-Aguas.

Bylander, T. (1994). The computational complexity of propositional STRIPS planning. *Artificial Intelligence*, 69(1–2):165–204.

Crosby, M., Jonsson, A., and Rovatsos, M. (2014). A single-agent approach to multiagent planning. In *Proceedings of the 21st European Conference on Artificial Intelligence (ECAI)*, volume 263 of *Frontiers in Artificial Intelligence and Applications*, pages 237–242. IOS Press.

Helmert, M. (2006). The Fast Downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246.

Helmert, M. (2009). Concise finite-domain representations for PDDL planning tasks. *Artificial Intelligence*, 173(5–6):503–535.

Howey, R., Long, D., and Fox, M. (2004). VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL. In *Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, pages 294–301. IEEE Computer Society.

Lotinac, D. and Jonsson, A. (2016). Constructing hierarchical task models using invariance analysis. In *Proceedings of the 22nd European Conference on Artificial Intelligence (ECAI)*, pages 1274–1282. IOS Press.

Nebel, B., Dimopoulos, Y., and Koehler, J. (1997). Ignoring irrelevant facts and operators in plan generation. In *Proceedings of the 4th European Conference on Planning (ECP)*, pages 338–350. Springer.

Richter, S. and Westphal, M. (2010). The LAMA planner: Guiding cost-based anytime planning with landmarks. *Journal of Artificial Intelligence Research*, 39:127–177.